Automating Data Import and Relationship Mapping Using Import Sets and Dot Walking

1. Ideation Phase

Objective: Identify the problem, explore solutions, and conceptualize the project.

Problem Statement:
Manual data imports into ServiceNow tables are time-consuming, prone to human error, and difficult to maintain when relationships between tables need to be established. For example, importing employee data and linking it to departments manually can result in inconsistencies.

Proposed Solution:
Automate the data import process using **Import Sets** and maintain relationships using **Dot-Walking**.

Key Features of the Solution:

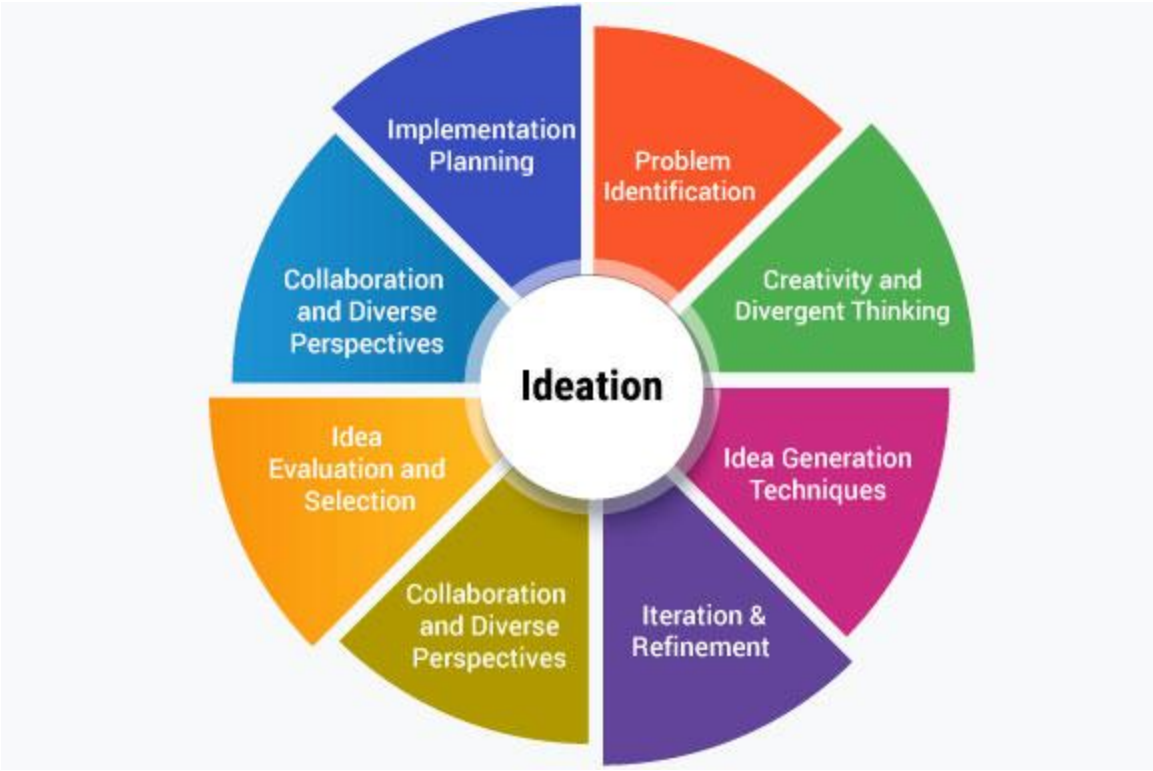
- Automates bulk data imports from CSV or Excel files.
- Maintains relational data integrity using Dot-Walking.
- Validates and transforms data during import to meet business rules.
- Provides error logging for failed imports.

Benefits:

- Reduces manual workload.
- Minimizes data entry errors.
- Ensures consistent relational mapping.
- Scalable for multiple datasets and tables.

Tools Used:

- ServiceNow Instance
- Import Sets and Transform Maps
- GlideRecord scripting (for complex relationships)



---

2. Request Analysis Phase

Objective: Gather requirements, define scope, and understand limitations.

Stakeholders:

- IT Administrators
- Data Analysts
- Business Managers

Functional Requirements:

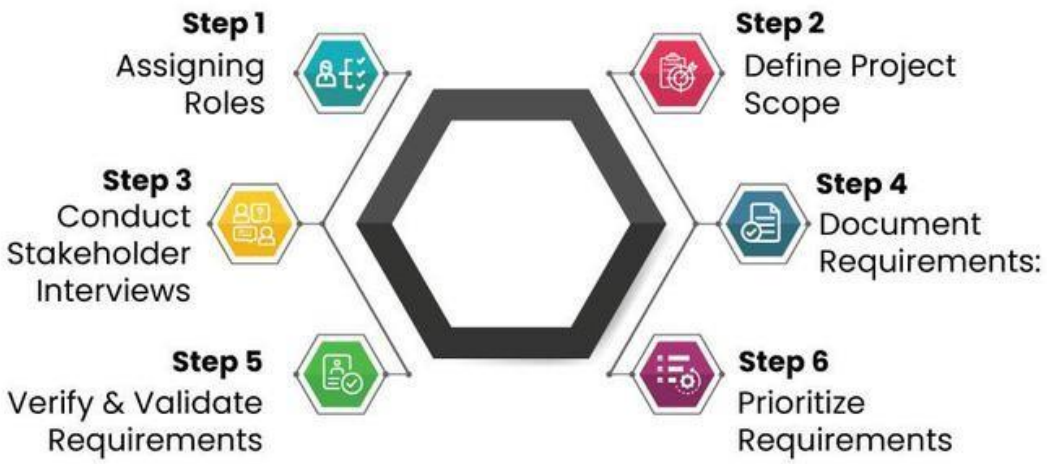
1. Upload and import datasets (CSV/Excel) into ServiceNow.
2. Maintain relationships between tables (e.g., employees → departments).
3. Validate data before import (mandatory fields, correct formats).
4. Log errors for failed records.

Non-Functional Requirements:

- Performance: Ability to handle up to 10,000 records per import.
- Security: Only authorized users can import data.
- Scalability: Support multiple import sets simultaneously.

Constraints:

- Limited to ServiceNow's Import Set and Transform Map capabilities.
- Must conform to existing ServiceNow table structures.



---

3. Project Planning Phase

Objective: Plan the project timeline, resources, and deliverables.

Resources Required:

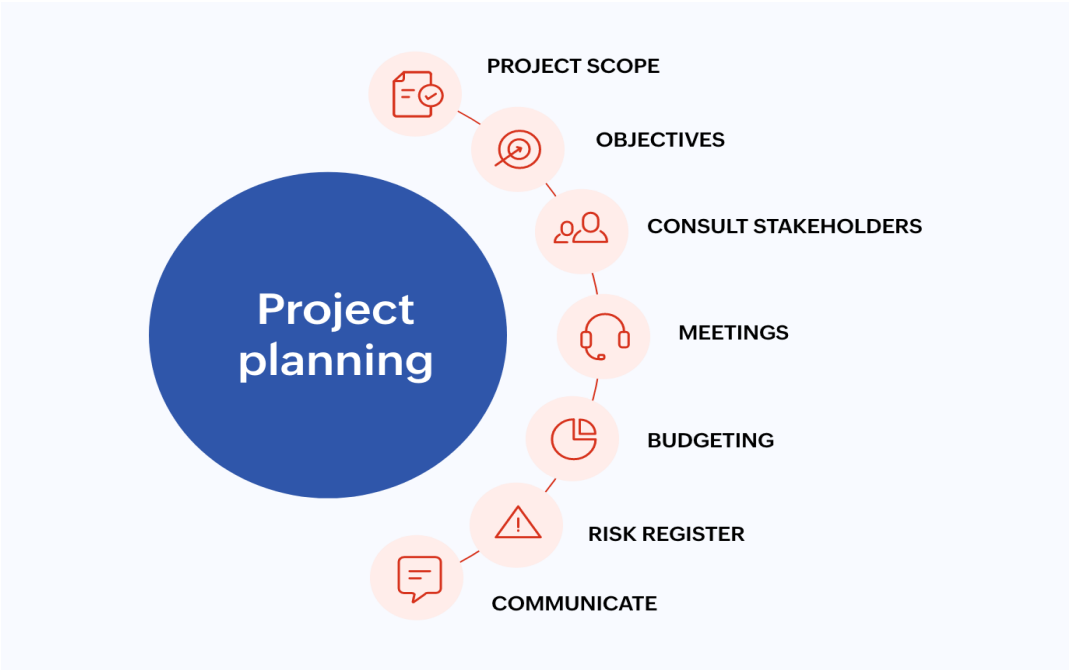
- ServiceNow Developer Instance
- Sample Datasets (Employee, Department, Asset Data)
- Team:
  - 1 Developer
  - 1 Tester
  - 1 Business Analyst

Project Timeline:

Phase	Duration	Activities
Ideation & Request Analysis	1 week	Problem definition, requirements gathering
Design	1 week	Data flow diagrams, table design, mapping strategy
Development	2 weeks	Import set configuration, transform map, scripting
Testing & Debugging	1 week	Sample data imports, error resolution
Report & Documentation	1 week	Final report, presentation, and demo preparation

Deliverables:

- Fully functional data import solution
- Documentation of mappings, scripts, and workflows
- Demo presentation showcasing working import and relational mapping



---

4. Project Design Phase

Objective: Define architecture, workflows, and data mappings.

System Architecture:

```
CSV/Excel Data → Import Set Table → Transform Map → Target Table
                                                  ↘ Dot-Walking → Related Tables
```

Key Design Considerations:

- Import Set Table: Temporary storage for raw data.
- Transform Map: Maps fields from Import Set Table to Target Table.
- Dot-Walking: Used to traverse relationships between tables (e.g., Employee → Department).
- Error Handling: Records failing validation are logged for correction.

Sample Data Flow:

Source Table	Import Field	Target Table	Target Field	Notes
Employee.csv	emp_id	Employee	Employee ID	Primary key
Employee.csv	emp_name	Employee	Name	Required
Employee.csv	dept_name	Department	Name	Use Dot-Walking to link
Employee.csv	manager_id	Employee	Manager Reference	Dot-Walk to Employee table

Dot-Walking Example:

- `employee.department.name` → Links Employee record to Department table using Department Name.

Validation Rules:

- Mandatory fields cannot be blank
- Unique constraints checked (e.g., Employee ID must be unique)
- Error handling logs failed rows

---

5. Project Development Phase

Objective: Implement the solution and automate the workflow.

Development Steps:

1. **Create Import Set Table:** Define temporary table structure matching CSV columns.
2. **Configure Transform Map:** Map import fields to target table fields.

3. **Implement Dot-Walking:** Establish relationships (e.g., Employee → Department).
4. **Data Transformation Scripts:** Use scripting for complex data conversions (e.g., date formats).
5. **Testing:** Import sample datasets to validate mappings.
6. **Error Logging:** Capture failed records in error logs for review.

Sample Script (Dot-Walking Example):

```
var empGR = new GlideRecord('u_employee');
empGR.initialize();
empGR.name = source.u_name;
empGR.department = source.u_department.getRefRecord().getUniqueValue();
empGR.insert();
```

Testing Scenarios:

- Correct dataset import
- Missing mandatory fields
- Invalid relationships
- Duplicate records

---

6. Project Report Phase

Objective: Document outcomes, challenges, and future enhancements.

Achievements:

- Successfully automated bulk data import.
- Established accurate relationships between tables using Dot-Walking.
- Reduced manual data entry and errors.

Challenges Faced:

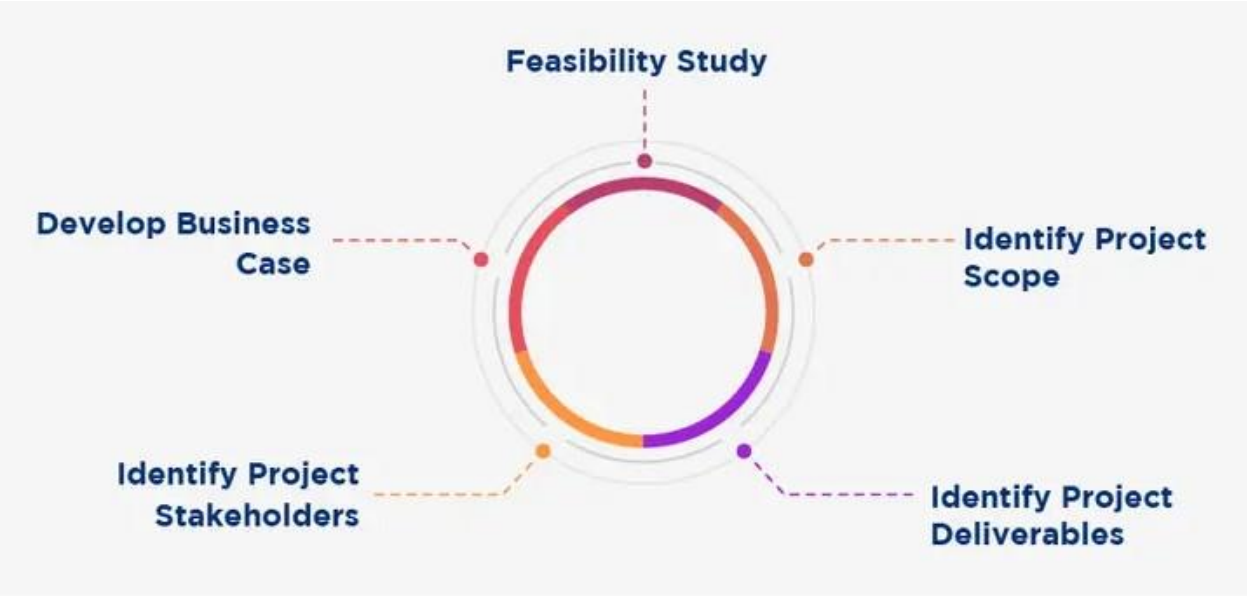
- Handling large datasets required batch imports to avoid timeout.
- Complex relationships needed multiple Dot-Walking configurations.

Future Enhancements:

- Automate scheduled imports.
- Support additional formats like JSON and XML.
- Add a user-friendly interface for mapping selection.

Screenshots/Diagrams:

- Import Set configuration
- Transform Map with Dot-Walking
- Sample imported data in target tables



---

7. Project Demonstration Phase

Objective: Present the working solution to stakeholders.

Demo Steps:

1. Upload a sample CSV file to Import Set Table.
2. Execute the Transform Map with Dot-Walking.
3. Display imported data in the target table.
4. Show relationships (e.g., Employee → Department) are correctly established.
5. Demonstrate error handling with invalid/missing data.

Expected Outcome:

- All records imported automatically.
- Relationships correctly mapped.
- Errors captured and logged for review.

Conclusion:
This project successfully automates data import while maintaining relational integrity, reducing manual work, and providing a scalable solution for ongoing data management in ServiceNow.